Step 16 - Private Peripheral Interrupts (PPIs) 1
Reference Note 1
Learning Objectives 1
1. What is a PPI? 1
2. Types of Interrupts in the GIC 2
Example 2
3. Why Do Timers Use PPIs? 2
4. Timer Interrupt Flow 3
Timer Path 3
5. Generic Timer PPIs 4
6. Physical Timer Interrupt 4
Example 4
7. Virtual Timer Interrupt 5
Usage 5
8. Hypervisor Timer Interrupt 6
Usage 6
9. Secure Physical Timer Interrupt 6
Usage 7
10. Why Not Use SPIs? 7
Example 8
Desired Behavior 8
11. Per-Core Timer Model 8
Key Observation 8
Architecture Summary 9
Common Misconceptions 9
Misconception 1 9
Misconception 2 9
Misconception 3 10
Misconception 4 10

Key Takeaways 10
Knowledge Check 10
Q1 11
Q2 11
Q3 11
Q4 11
Q5 11
Answer Key 11

Step 16 - Private Peripheral Interrupts (PPIs)

Reference Note

This chapter is based on the ARM Generic Timer Architecture and ARM Generic Interrupt Controller (GIC) architecture concepts.

In Step 15, we learned that a timer expires when:

Current Count >= Compare Value

and the timer asserts its output.

This chapter explains:

How does timer expiry become a CPU interrupt?
Why does each core have its own timer interrupt?
What is a PPI?

Learning Objectives

After completing this chapter, you should understand:

- What a PPI is
- Why ARM timers use PPIs
- Relationship between timers and CPU cores
- Physical Timer interrupt
- Virtual Timer interrupt
- Hypervisor Timer interrupt
- Secure Timer interrupt
- Why timer interrupts are private to a core

1. What is a PPI?

PPI stands for:

Private Peripheral Interrupt

A PPI is:

An interrupt that is private to a specific CPU core.

Unlike shared interrupts, a PPI belongs to each processor independently.

2. Types of Interrupts in the GIC

Interrupt Type	Scope
SGI	Software Generated Interrupt
PPI	Private to a CPU Core
SPI	Shared Peripheral Interrupt

Example

Interrupt	Type
Timer Interrupt	PPI
Ethernet Controller	SPI

UART	SPI
CPU-to-CPU Signal	SGI

3. Why Do Timers Use PPIs?

Each CPU core owns its own timers.

Example:

Core	Timer
Core 0	Physical Timer
Core 1	Physical Timer
Core 2	Physical Timer
Core 3	Physical Timer

Each timer should interrupt:

Its own CPU

not:

Any CPU

Therefore ARM uses PPIs.

4. Timer Interrupt Flow

When the timer expires:

Step	Action
1	Counter reaches compare value
2	Timer expires
3	Timer output asserted
4	PPI becomes pending
5	GIC signals CPU

6	CPU enters ISR

Timer Path

Component	Purpose
Counter	Measures time
Timer	Detects expiry
PPI	Interrupt source
GIC	Interrupt delivery
CPU	Executes ISR

5. Generic Timer PPIs

The ARM architecture defines PPIs for Generic Timer events.

Common timer interrupt sources are:

Timer	Typical PPI
Secure Physical Timer	PPI 29
Non-secure Physical Timer	PPI 30
Virtual Timer	PPI 27
Hypervisor Timer	PPI 26

Actual routing can vary by implementation and security configuration.

6. Physical Timer Interrupt

Source:

Physical Timer

Registers:

CNTP_CVAL_EL0
CNTP_TVAL_EL0
CNTP_CTL_EL0

Interrupt Target:

Own CPU Core

Example

Core	Physical Timer Interrupt
Core 0	PPI 30
Core 1	PPI 30
Core 2	PPI 30
Core 3	PPI 30

Notice:

Same PPI Number
Different CPU Instance

Each core receives its own PPI.

7. Virtual Timer Interrupt

Source:

Virtual Timer

Registers:

CNTV_CVAL_EL0
CNTV_TVAL_EL0
CNTV_CTL_EL0

Interrupt:

Virtual Timer PPI

Typically:

PPI 27

Usage

Commonly used by:

Software
Guest Operating System
Virtual Machines

8. Hypervisor Timer Interrupt

Source:

Hypervisor Timer

Registers:

CNTHP_CVAL_EL2
CNTHP_TVAL_EL2
CNTHP_CTL_EL2

Interrupt:

Hypervisor Timer PPI

Typically:

PPI 26

Usage

Purpose
VM Scheduling
Hypervisor Services
Maintenance Tasks

9. Secure Physical Timer Interrupt

Source:

Secure Physical Timer

Registers:

CNTPS_CVAL_EL1
CNTPS_TVAL_EL1
CNTPS_CTL_EL1

Interrupt:

Secure Timer PPI

Typically:

PPI 29

Usage

Software
Secure Monitor
Trusted Firmware
Secure Services

10. Why Not Use SPIs?

Suppose timers used:

Shared Peripheral Interrupts

Problem:

Which CPU should receive the timer interrupt?

The timer belongs to one CPU.

Therefore:

Private Interrupt

is the correct design.

Example

Desired Behavior

Core	Receives Interrupt
Core 0 Timer	Core 0
Core 1 Timer	Core 1
Core 2 Timer	Core 2
Core 3 Timer	Core 3

PPIs provide exactly this behavior.

11. Per-Core Timer Model

Resource	Per Core?
System Counter	No

Physical Timer	Yes
Virtual Timer	Yes
Hypervisor Timer	Yes
PPIs	Yes

Key Observation

ARM uses:

Component	Sharing Model
System Counter	Shared
Timers	Per Core
PPIs	Per Core

This design scales naturally to multicore systems.

Architecture Summary

Component	Scope
System Counter	Shared System Resource
Physical Timer	Per CPU
Virtual Timer	Per CPU
Hypervisor Timer	Per CPU
Interrupt Delivery	Per CPU via PPI

Common Misconceptions

Misconception 1

PPI means one interrupt for the whole system

False.

A PPI is private to each CPU.

Misconception 2

All cores share one Physical Timer

False.

Each core has its own Physical Timer.

Misconception 3

Timers should use SPIs

False.

Timer events belong to individual CPUs.

Misconception 4

Shared Counter means shared timer

False.

The counter is shared.

The timers are per-core.

Key Takeaways

Concept	Key Point

PPI	Private Peripheral Interrupt
Purpose	Per-core interrupt delivery
Physical Timer	Typically PPI 30
Virtual Timer	Typically PPI 27
Hypervisor Timer	Typically PPI 26
Secure Timer	Typically PPI 29
Shared Resource	System Counter
Private Resource	Timers and PPIs

Knowledge Check

Q1

What does PPI stand for?

Q2

Why do Generic Timers use PPIs?

Q3

Is the System Counter shared or private?

Q4

Are timers shared or private to a core?

Q5

Why would an SPI be a poor choice for timer interrupts?

Answer Key

Question	Answer

Q1	Private Peripheral Interrupt
Q2	Timer events belong to a specific CPU core
Q3	Shared
Q4	Private per core
Q5	It would be unclear which CPU should receive the interrupt